CS-107 Lecture 1+2
The Big Picture

Troy Astarte

Megan Venn-Wycherley

Before we begin

- **Serious Nut Allergy in the department**
  - Please do not consume any nut-based products during lectures/labs/using teaching spaces etc.

Introduction

Goals for Today

- Brief introduction to the field of Computer Science
- **Layers** of a computer system and the concept of **abstraction** and its relationship to computing
- History of computer **hardware** and **software**
- Looking forward

What is a Computer?

- "One who computes; a calculator, reckoner; *spec.* a person employed to make calculations in an observatory, in surveying, etc." - (used in 1613)
- "A device or machine for performing or facilitating calculation." - Oxford English Dictionary (used in 1869)
- "An electronic device (or system of devices) which is used to store, manipulate, and communicate information, perform complex calculations, or control or regulate other devices or machines" - Oxford English Dictionary (used in 1945)

What is Computer Science?

- Computing Science is more than just the computer.
- “The branch of knowledge concerned with the construction, programming, operation, and use of computers.” - Oxford English Dictionary (used in 1956)
- It is hardware, software, and the users interacting with them. It encompasses mathematics, electronics, psychology, engineering, and many more wider fields…

Hardware vs Software

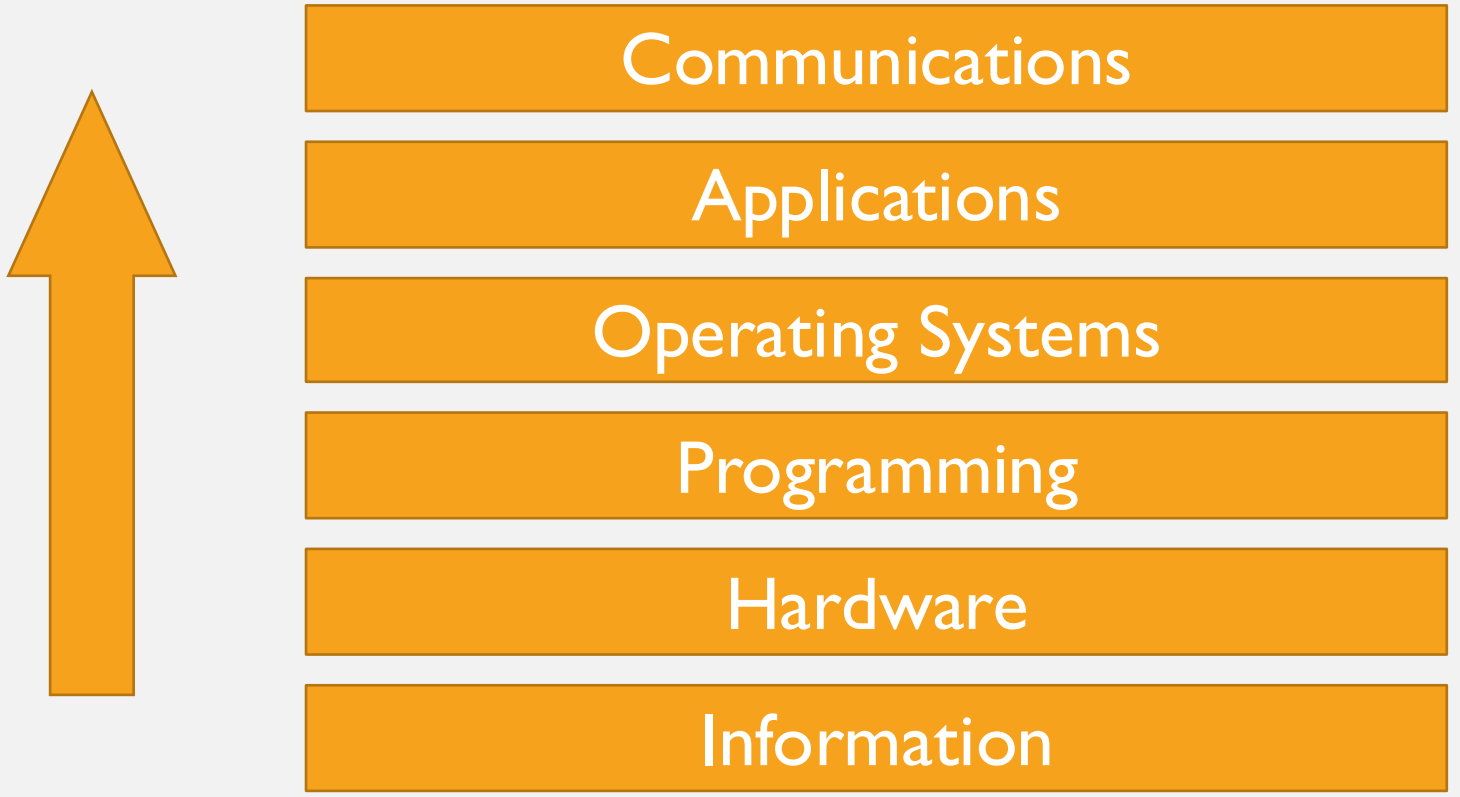
Hardware

- The physical elements of a computing system – printer, circuit boards, wires, keyboard, monitor, etc.

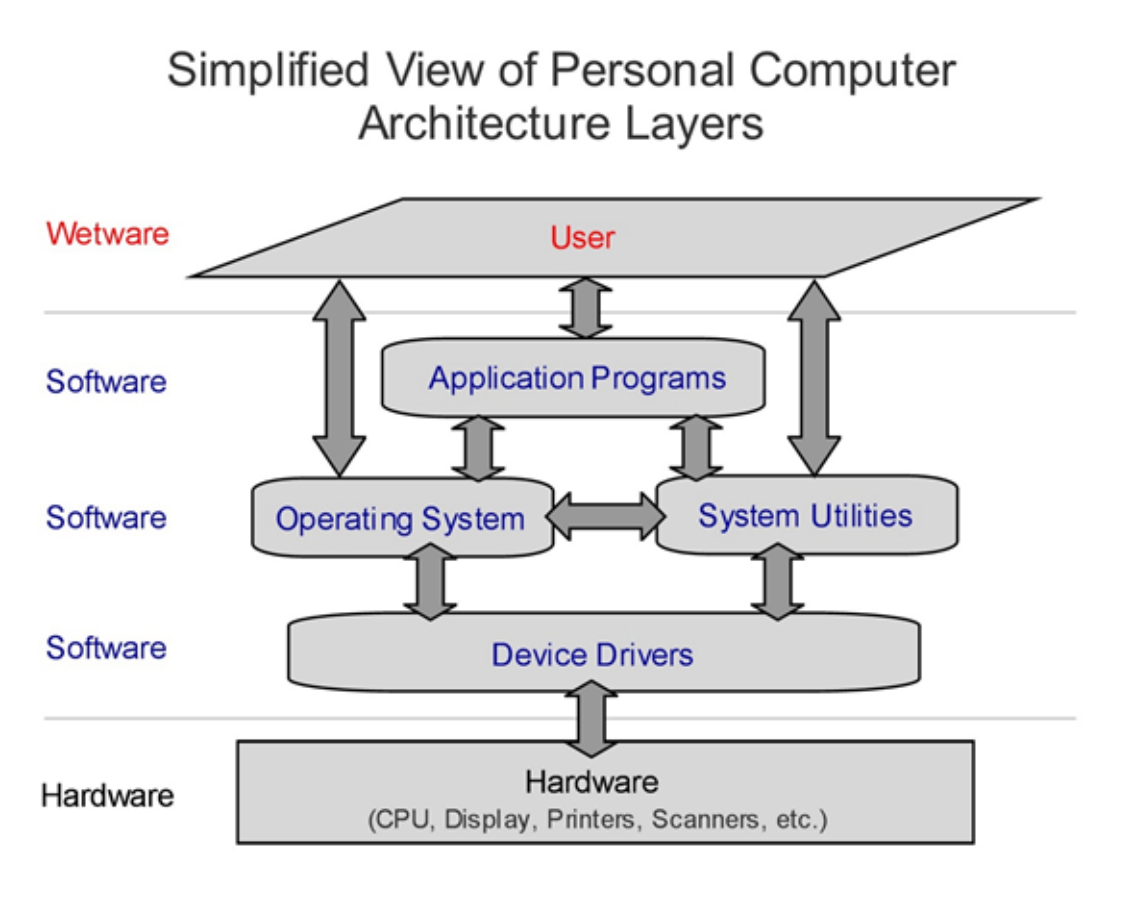
Software

- The programs that provide the instructions for a computer to execute.

The Layers


Communications
Applications
Operating Systems
Programming
Hardware
Information

The Layers: Another Perspective



Central image from https://computerbasicsforluddites.wordpress.com/2013/10/13/pc-software-and-hardware-video/

Abstraction

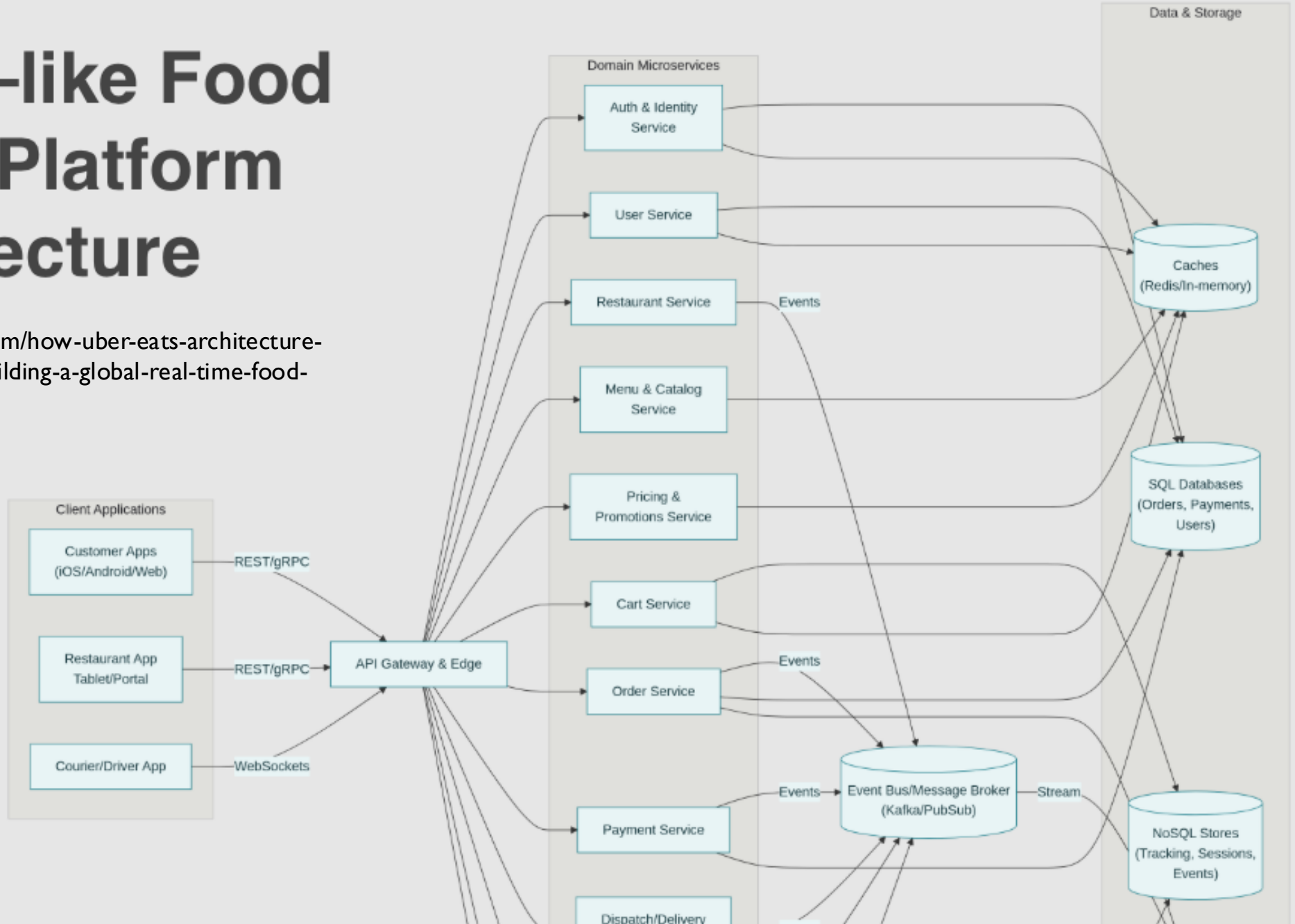


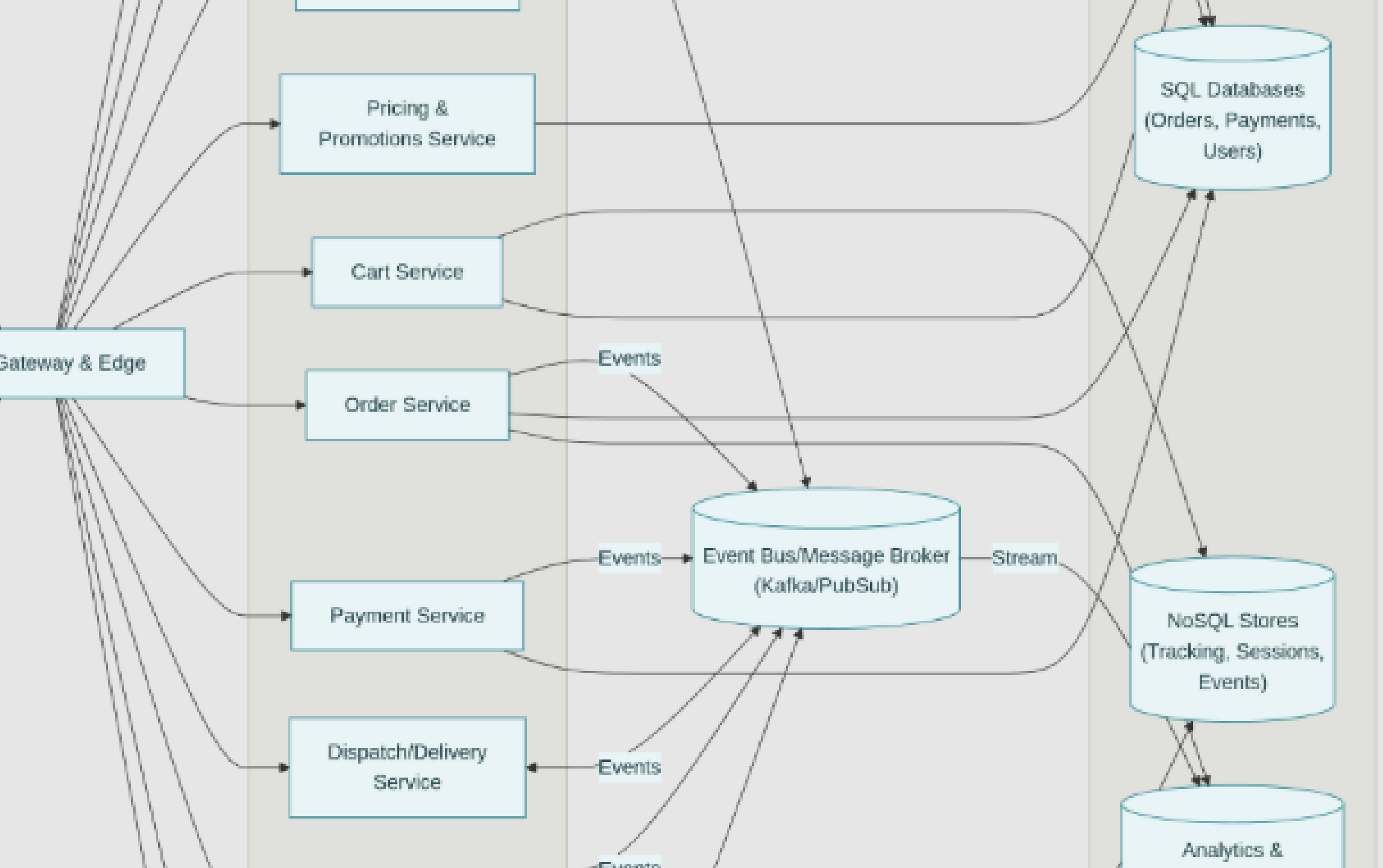
I was preparing these slides, and all I wanted was a bacon cheeseburger. I open up my app, place my order, and twenty minutes later I got my cheeseburger.

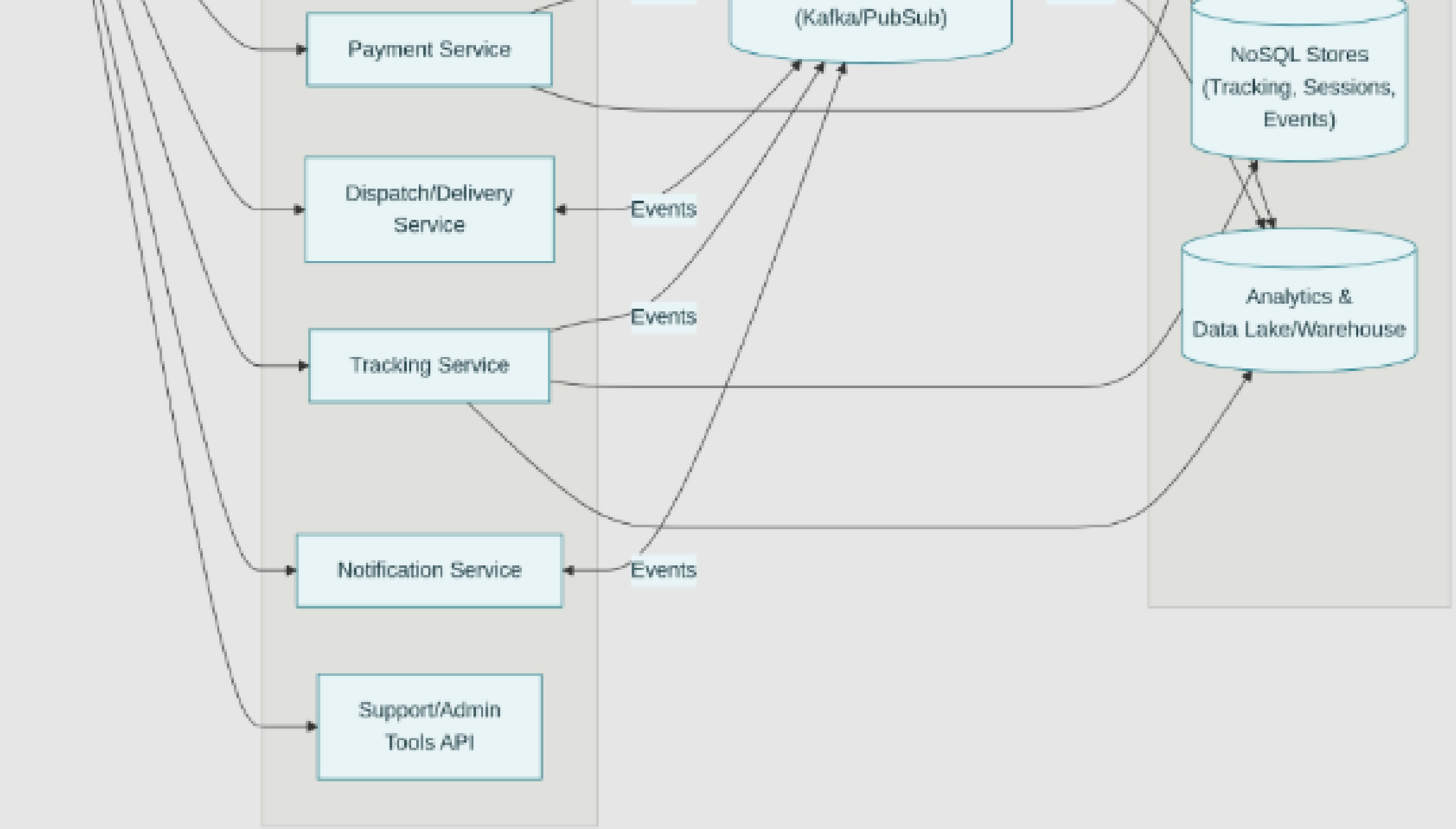
How is this abstraction?

Uber Eats–like Food Delivery Platform Architecture

https://blog.stackademic.com/how-uber-eats-architecture-works-a-deep-dive-into-building-a-global-real-time-food-delivery-222dde27666f




Pricing &
Promotions Service
SQL Databases
(Orders, Payments,
Users)
Cart Service
Gateway & Edge
Order Service
Events
Event Bus/Message Broker
(Kafka/PubSub)
Events
Stream
Payment Service
NoSQL Stores
(Tracking, Sessions,
Events)
Dispatch/Delivery
Service
Events
Analytics &


Payment Service
(Kafka/PubSub)
NoSQL Stores
(Tracking, Sessions,
Events)
Dispatch/Delivery
Service
Events
Analytics &
Data Lake/Warehouse
Events
Tracking Service
Notification Service
Events
Support/Admin
Tools API

Abstract vs Concrete

- **Abstraction** is nice, it allows us to avoid dealing with the lower level details when the task doesn't require us to.

- However at some point, somebody will need to deal with that detail. This is where the idea becomes **concrete** and is **realised** in some form.

- For example; the **abstracted** app is handled by the **concrete** building of databases, API calls and client facing apps.

The Big Picture

- Being able to understand the big picture of computing means considering the properties of a complex system as a whole, not just the part you can see.

- A delivery app is an interface, a network, servers, databases and payment systems working together: change one and the others will be impacted.

Where did Computers come from?

Agriculture?

Around **12,000 years ago**, humans began switch from hunter-gatherer lifestyles to permanent farming

As techniques improved, produce became more than needed – trade began to be developed

And **money**

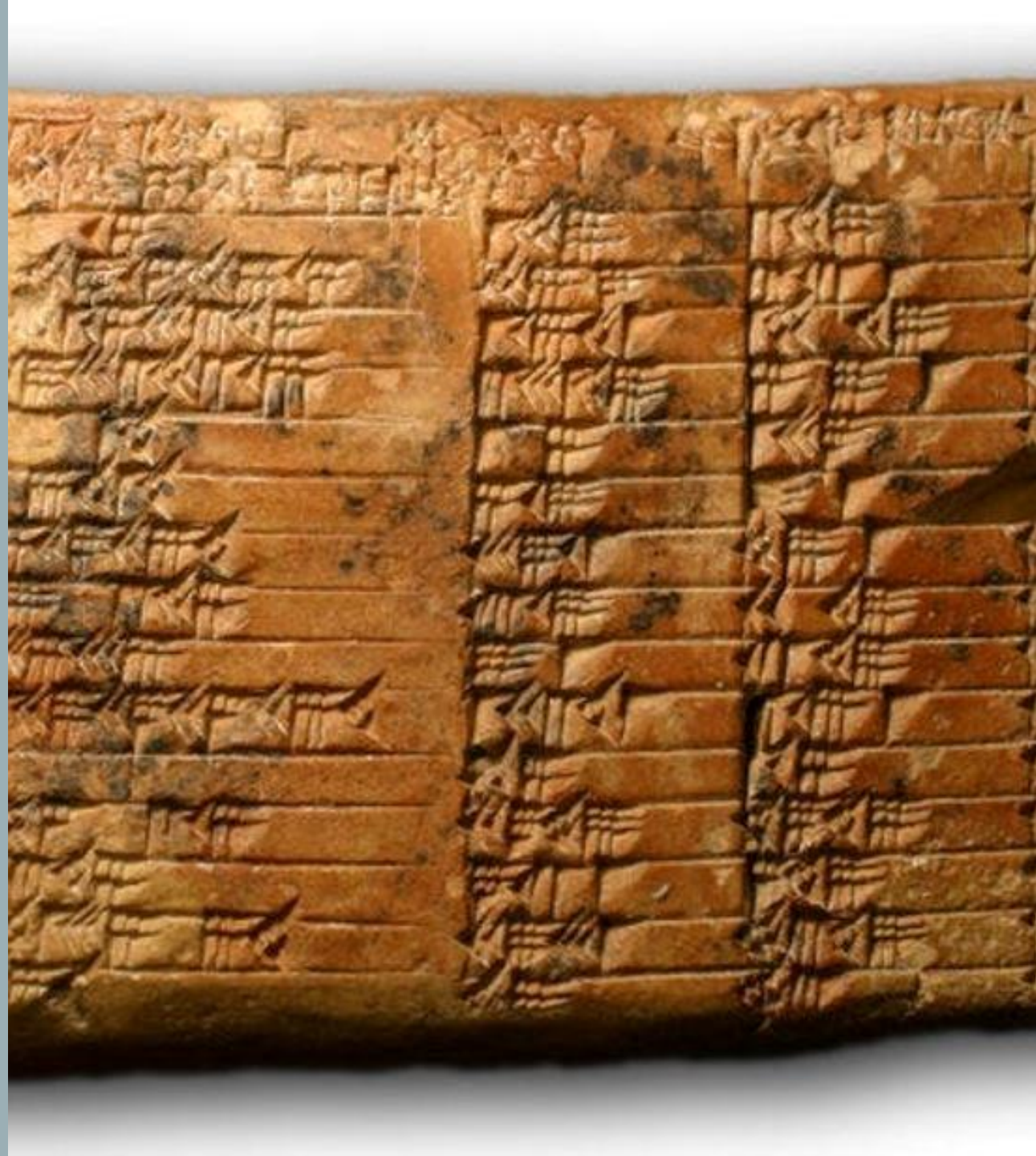


Manual Calculation

First appeared between 2700 and 2300BCE in Sumer (modern day Southern Iraq)

Simple, stone tablets divided into 5 columns: 1s, 10s, 60s, 600s, and 3600s

The abacus addressed need for computation of large numbers due to a growing population and economic system



Trade?

You've got goods, and you've got money – but nobody is buying nearby. Now you've got to travel further (potentially across the sea)

How do you get there, and back again, safely?

Sailing – **and the stars.**



Mechanical

Retrieved from a shipwreck in Antikythera 1900 - theorised to have set of from Rhodes, a busy trading port in 204BCE

Comprised 37+ bronze gears to follow movements of the sun/moon, track eclipses

The first **analogue** computer?



Taxation?

You've got goods, you can travel, you've created machines to automate jobs…

The population is booming, you need to care about government planning, military organisation and taxes.

How do you start **counting the population (so you can tax them..)?**



Tabulating

1880 Census took 8 years to process in US – wanted ways to improve.

Herman Hollerith, inspired by rail ticket punching and industrial weaving looms, designed census cards.



Warfare?

Now you've got international trade, improved military organisation, and economic recessions on a global scale

Leading to a need for **national defence and offense.**

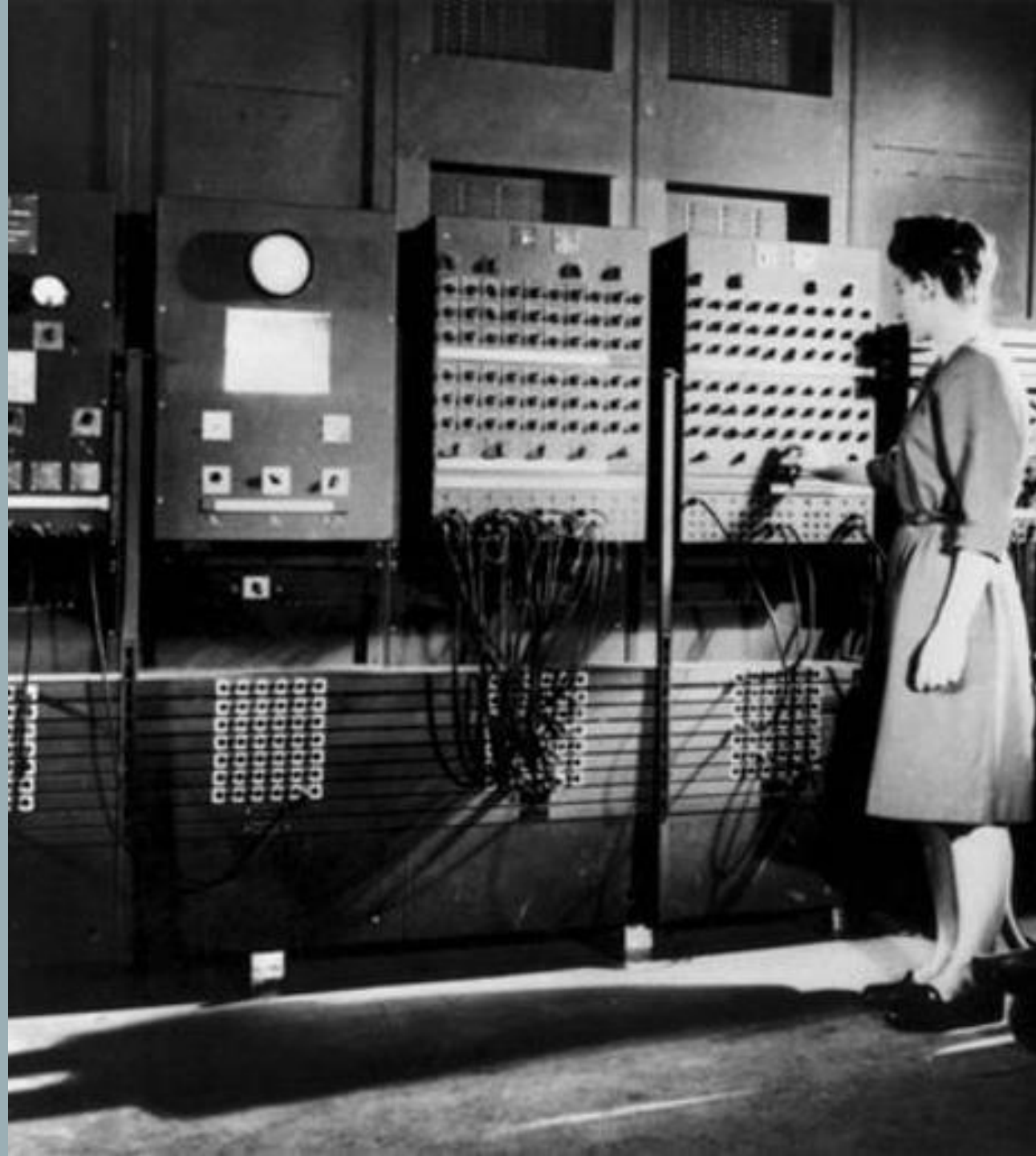


ENIAC

The US military needed to calculate complex ballistics tables during World War II – people make mistakes, manual methods too slow.

The **E**lectronic **N**umerical **I**ntegrator **A**nd **C**omputer (ENIAC) was a general-purpose electronic computer.

While it could calculate 5,000 additions per second, programming the ENIAC could take several days (and often meant moving wires).



DIGITAL COMPUTER HARDWARE (1950-1960)

Vacuum tubes

- Used for logical circuitry.
- Create a lot of heat, quite often fail.

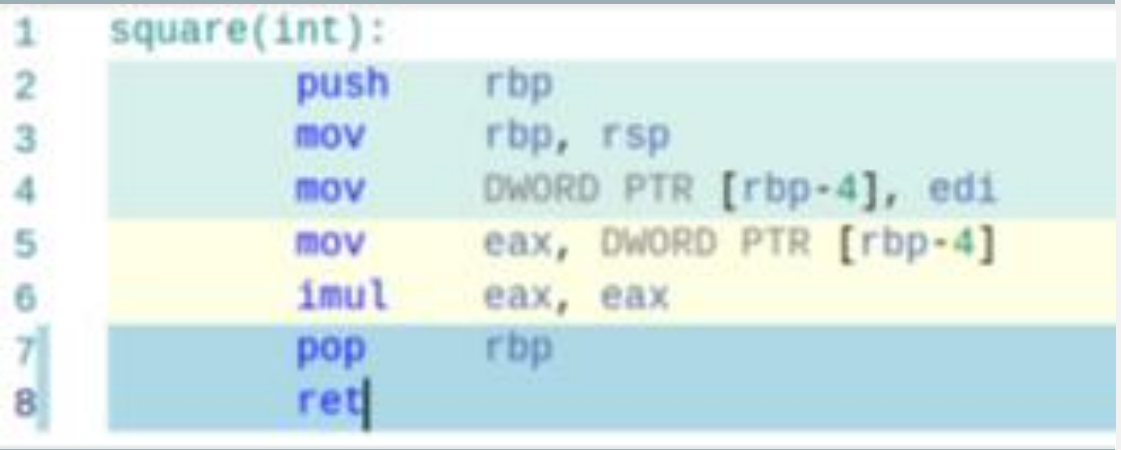
Magnetic drums

- Memory that rotates under a read/write head.
- Big, slow, expensive to run.

Card readers and magnetic tapes

- Used for storage of information (programs and data).

DIGITAL COMPUTER SOFTWARE (1950S)



Machine Code

- Programs written in binary (1s and 0s)

Assembly languages and translators

- Programs written using mnemonics
- Translated into machine code by assemblers
- Machine code is then run on the machine

DIGITAL COMPUTER HARDWARE (1960-1965)

Transistors

- Replaced vacuum tubes.
- Fast, small, durable, and cheap.

Magnetic cores

- Replaced magnetic drums.
- Information available instantly. Still takes up space.

Magnetic disks

- Replaced magnetic tape.
- Data can be accessed directly.

DIGITAL COMPUTER SOFTWARE (1960S)

High-Level Languages

- Programs written in more human-friendly languages.
- Early languages included:
- Fortran, Algol, COBOL and Lisp.

Compilers and Interpreters

- Programs written in higher levels…
- But still need to run on the machine!
- They can either be compiled into low level machine code, or run by an interpreting program.
-

DIGITAL COMPUTER HARDWARE (1965-1970)

Integrated Circuits

- Replaced circuit boards.
- Smaller, cheaper, faster and more reliable

Transistors

- Now used for constructing memory

Terminal

- Input/Output devices with a keyboard and screen

DIGITAL COMPUTER SOFTWARE (1970S)

Utilise layers to add abstraction between end users and the underlying machine

Explosion in the area of "applications" in recent years

- Growth in desktop and mobile applications
- Cloud-based computing
- Range of embedded systems

Want to know more?

- History of Computing Collection https://www.swansea.ac.uk/library/archive-and-research-collections/hocc

- There are some items found on both Singleton and Bay Campus, worth having a look

Why should we care?

- COBOL (1959) runs 95% of ATM transactions, $3t of daily bank transactions, 60m+ patient records…

- Being able to understand how the “black box” of technology works, avoiding design mistakes, writing efficient software and understanding vulnerabilities are all important computing skills

Data Representation and Computer Architecture

The Module

To understand "what can be efficiently computed" we need:

- Algorithmic thinking
- Data representation
- Programming
- Design

Topics

- From numbers to data
- Building the machine
- Getting the machine running
- Everyday use of computers
- Wider applications of modern computers

The Delivery: Lectures

Mondays: 2-4pm in Eng Central B004

Friday: 10-11am in Eng Central B004

Classic lecture style teaching. Introducing terminology and knowledge of the topic. Remember to take notes, not everything is on the slides!

The Delivery: Labs

Fridays

Labs 11-1pm in CoFo 104 (PC)

Hands-on sessions to reinforce theory. Can include practical examples, Q&As, programming exercises etc. Be sure to attend for a deeper appreciation and understanding of the topic!

The Delivery: Us

Troy and Megan

We share the module and alternate the sessions week by week.

Whichever of us is not teaching that week still holds drop-in hours, so bring questions to either of us.

Drop-In Hours

Megan

Mondays 10-11am, 4-5pm in CoFo 404

Troy

Tuesdays & Thursdays 10-11:00 CoFo 407

Opportunity to come ask questions, clarify understanding…

Assessment

- CW1 - From Numbers to Data & Building the Machine (15%) - October
- CW2 - Getting it Running and Everyday Use (20%) - November

- **1 x Exam (65%):**
- Sat in the January exam period.
- Everything on the module has chance to come up on the exam.

- **Make sure you do every piece of assessment!**

Our Expectations

Arrive on time and attend all lectures and workshops, they are designed to support you through the content!

Make the most of drop-in hours and ask questions, we're here to help

Flag problems early so we can point you in the right direction.

Your Expectations

We respond to your questions within 2 working days.

We are available during our drop-in hours, unless we have announced a disruption in advance.

We publish slides and lab material on Canvas so you can revisit anything you miss.

CS-107 Lecture 1+2
The Big Picture

Troy Astarte

Megan Venn-Wycherley